



Programacion Orientada a Objetos **2**

Ing. Jose Rusca
Ing. Federico Stulich
Ing. Francisco De Grandis



Clases en JavaScript



JS

Clase de hoy



1. Definición de clases
2. Creación de objetos
3. Metodos
4. Propiedades/métodos privados
5. Propiedades/métodos estáticos
6. Definición de sub-clase (herencia)
7. Polimorfismo

Clases en Javascript



Pero.....

¿no comentamos que javascript era un lenguaje basado en prototipos?

¿por que ahora volvemos a hablar de clases?

Repasemos un poco los conceptos.....

Clases en Javascript



JavaScript es un lenguaje de programación basada en prototipos, multiparadigma, de un solo hilo, dinámico, con soporte para programación orientada a objetos, imperativa y declarativa.

Clases en Javascript



Entonces... ¿Qué es la programación basada en prototipos?

La programación basada en prototipos es un estilo de programación orientada a objetos en donde las clases no se definen explícitamente, sino que se derivan de agregar propiedades y métodos a una instancia de otra clase o, con menos frecuencia, agregarlos a un objeto vacío.

En palabras simples: este tipo de estilo permite la creación de un objeto sin definir primero su clase.....

Volveremos más adelante con la programación basada en prototipos.

Clases en Javascript



Las clases de javascript, son una mejora sintáctica sobre la herencia basada en prototipos de JavaScript.

La sintaxis de las clases no introduce un nuevo modelo de herencia orientada a objetos en JavaScript.

Las clases de JavaScript proveen una sintaxis mucho más clara y simple para crear objetos y lidiar con la herencia.

Clases en Javascript



Por lo tanto, JavaScript permite una sintaxis para definir clases pero en realidad es una carcasa y traduce esto a prototipos.

La idea de todo esto fue poner javascript al alcance de todos aquellos programadores que vienen de la POO clásica definiendo clases.

Como nosotros también venimos del mundo de las clases vamos a tomar este camino... y luego iremos por los prototipos.

Definición de una clase



En los lenguajes basados en clases las **clases** se definen de forma separada.

En esa definición se puede especificar métodos especiales, llamados **constructores**, para crear instancias de la clase.

Un método **constructor** puede especificar valores iniciales para las propiedades de la instancia y realizar otro procedimiento de inicialización apropiado en el momento de la creación.

Se utiliza el operador **new** junto al constructor para crear instancias de clases.

Definición de una clase

```
class Persona {  
    constructor(nombre, apellido) {  
        this.nombre = nombre;  
        this.apellido = apellido;  
    }  
}  
  
let javy = new Persona("Javier", "Baez");
```

Propiedades

Definición de una clase

```
class Persona {  
    constructor(nombre, apellido) {  
        this.nombre = nombre;  
        this.apellido = apellido;  
    }  
}  
  
let javy = new Persona("Javier", "Baez");
```

No debemos olvidar el **this**.
Si no vamos a estar
asignando el valor a la
misma variable.



Definición de una clase

```
class Persona {  
    constructor(nombre, apellido, edad) {  
        this.nombre = nombre;  
        this.apellido = apellido;  
        this.edad = edad;  
    }  
    ...  
}
```

Supongamos que necesitamos la edad de la persona.

Procedemos a modificar el constructor y agregar la propiedad.

Definición de una clase



Agregando la edad nos vamos a encontrar, que en un año o menos, ese valor va a cambiar.

Por lo tanto sería mejor almacenar la fecha de nacimiento y calcular la edad al momento de necesitar el valor.

Definición de setters y getters

```
class Persona {  
    constructor(nombre, apellido, fechaNacimiento) {  
        this.nombre = nombre;  
        this.apellido = apellido;  
        this.fechaNacimiento = fechaNacimiento;  
    }  
    ...  
}
```



Como visualizar las variables globales

```
function varList() {  
    return Object.getOwnPropertyNames(this);  
}
```

```
console.log(varList());
```

```
//También podemos visualizarlas en el modo DEBUG
```

```
//del Visual Code
```



Definición de un método



En JavaScript, los métodos son las propiedades de un objeto que contienen una función.

O también podemos decir que, los métodos son funciones almacenadas como una propiedad.

Definición de un método

```
class Persona {  
  
    constructor(nombre, apellido) {  
  
        this.nombre = nombre;  
  
        this.apellido = apellido;  
  
    }  
  
    nombreCompleto() {  
  
        return this.apellido + ", " + this.nombre;  
  
    }  
  
}
```



Propiedades/métodos públicos y privados



En JavaScript todos los campos (propiedades) son públicos.

Recientemente el standard de ECMAScript agregó la posibilidad de definir campos privados anteponiendo # (hash) al nombre de la propiedad.

Propiedades/métodos públicos y privados

```
class Persona {  
    #dni;  
    constructor(nombre, apellido, dni) {  
        this.nombre = nombre;  
        this.apellido = apellido;  
        this.#dni = dni;  
    }  
}
```



```
let javy = new Persona("Javier", "Baez", 12345678);
```

El campo no puede ser
accedido directamente.



Propiedades/métodos estáticos



La palabra clave **static** define un método o propiedad estática para una clase.

No se pueden invocar métodos estáticos ni propiedades estáticas en objetos (instancias de clase).

Se los debe llamar desde la clase misma.

Propiedades/métodos estáticos

```
class Persona {
```

```
    static total=0;
```

Propiedad estática.

```
    constructor(nombre, apellido, fechaNacimiento, dni) {
```

```
        ... // Asignar valores a propiedades
```

```
        Persona.total=Persona.total+1;
```

```
    }
```

```
    static totalPersonas() {
```

```
        return Persona.total;
```

```
}
```



Propiedades/métodos estáticos

```
class Persona {  
    static total=0;  
    constructor(nombre, apellido, fechaNacimiento, dni) {  
        ... // Asignar valores a propiedades  
        Persona.total=Persona.total+1;  
    }  
    static totalPersonas() {  
        return Persona.total;  
    }  
}
```

El método estático no tiene por qué estar asociado a una propiedad estática, en el ejemplo lo usamos para mostrar un valor estático pero podría ser cualquier otro valor.

Definición de setters y getters



Las propiedades definidas de esta forma se conocen como propiedades de acceso y podemos distinguirlas fácilmente de las otras propiedades que tienen un simple valor.

Get: Enlaza la propiedad de un objeto con una función que será llamada cuando la propiedad es buscada.

Set: Asocia la propiedad de un objeto a una función que será llamada cuando haya un intento de asignar valor a esa propiedad.

Definición de setters y getters

 **No utilizar**

```
class ClaseQueUtilizaGetSet {  
  #msg = "hello world";  
  get msg() { return this.#msg;}  
  set msg(x) {this.#msg = `hello ${x}`;}  
}  
  
const instance = new ClassWithGetSet();  
console.log(instance.msg); // "hello world"  
instance.msg = "cake";  
console.log(instance.msg); // "hello cake"
```



Definición de setters y getters



No utilizar

```
class Persona {  
    ....  
    get edad() {  
        return parseInt((new Date() - this.fechaNacimiento)  
                           /31536000000);  
    }  
}  
  
let javy = new Persona("Javier", "Baez", new Date(1992,12,1));  
console.log(javy.edad)
```



Definición de una sub-clase



En una definición de clase, puedes especificar que la nueva clase es una **subclase** de una **clase existente**. Esta subclase hereda todas las propiedades de la superclase y puede añadir nuevas propiedades o modificar las heredadas.

Definición de una sub-clase

```
class Empleado extends Persona {  
    constructor(nombre, apellido, fechaNacimiento, sueldo) {  
        super(nombre, apellido, fechaNacimiento);  
        this.sueldo = sueldo;  
    }  
  
    detalleSueldo() {  
        return this.apellido + " cobra: " + this.sueldo;  
    }  
}
```



Ejercicio Polimorfismo



- Empleados
- Movimientos de Ajedrez

Referencia

- <https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Classes>
- <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/constructor>
- <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/static>
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/Public_class_fields
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/Private_class_fields

Próxima Clase



Programación orientada a prototipos

JavaScript

